

Aula 4 - Java e POO Avançada: SOLID, Design Patterns e Boas Práticas

Duração: 2 horas

Objetivos:

- Dominar os princípios SOLID na prática
- Implementar Design Patterns (Singleton, Factory, Strategy)
- Aplicar boas práticas de Orientação a Objetos

Parte Teórica (40 min)

1. Princípios SOLID

Princípio	Definição	Exemplo de Violação → Correção
SRP	Uma classe deve ter uma única responsabilidade	Classe "Usuario" que salva no BD e valida o email Separar em Usuario e UsuarioRepo
OCP	Aberto para extensão, fechado para modificação	if-else para novos tipos → Usar herança/interfaces
LSP	Subclasses devem substituir suas classes base	Pinguim herdar de Ave que tem voar Criar interface Voador
ISP	Múltiplas interfaces específicas são melhores que uma geral	Interface com 20 métodos → Dividir em Logavel, Salvavel
DIP	Depender de abstrações, não de implementações	new MySqlConnection() → Usar interface DatabaseConnection

2. Design Patterns

- **Singleton:** Garante uma única instância (ex: conexão com BD)
- **Factory:** Centraliza criação de objetos (ex: DocumentoFactory.createPDF())
- **Strategy:** Algoritmos intercambiáveis (ex: PagamentoStrategy: Credito, Boletto)

3. Boas Práticas em POO

- Prefira composição em vez de herança
- Encapsulamento: atributos `private` + getters/setters
- Evite acoplamento alto (`new` dentro de classes → usar injeção de dependência)

Parte Prática (1h20min)

Atividade 1: Implementando SOLID

Código com problemas (viola SRP e OCP):

```
java

class Relatorio {
    void gerarRelatorio(String tipo) {
        if (tipo.equals("PDF")) {
            // Lógica complexa de PDF
        } else if (tipo.equals("HTML")) {
            // Lógica complexa de HTML
        }
    }

    void salvarNoBanco() {
        // Lógica de BD
    }
}
```

Exercício: Refatore aplicando:

1. SRP: Separar geração de relatório e persistência
2. OCP: Usar Strategy Pattern para novos formatos

Solução Base:

```
java

// Strategy Pattern
interface FormatoRelatorio {
    void gerar();
}

class PDF implements FormatoRelatorio { /*...*/ }
class HTML implements FormatoRelatorio { /*...*/ }
```

```
// SRP
class RelatorioService {
    private RelatorioRepository repository;

    void gerar(FormatoRelatorio formato) {
        formato.gerar();
    }
}
```

Atividade 2: Implementando Singleton e Factory

Cenário: Conexão com banco de dados (Singleton) + Factory para documentos.

1. Singleton (DatabaseConnection.java):

java

```
public class DatabaseConnection {
    private static DatabaseConnection instance;

    private DatabaseConnection() {}

    public static synchronized DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}
```

2. Factory (DocumentoFactory.java):

java

```
public class DocumentoFactory {
    public static Documento criarDocumento(String tipo) {
        return switch (tipo) {
            case "PDF" -> new PDF();
            case "DOCX" -> new Docx();
            default -> throw new IllegalArgumentException("Formato inválido");
        };
    }
}
```

Exercício:

- Implemente um `Logger` como Singleton
- Crie uma `PaymentFactory` para `CreditCard`, `Pix`

Desafio Prático (30 min)

Contexto: Refatore o código abaixo aplicando:

1. **Strategy Pattern** para cálculo de impostos (ICMS, ISS)
2. **DIP** para evitar acoplamento

java

```
class CalculadoraImposto {
    double calcular(String tipo, double valor) {
        if (tipo.equals("ICMS")) {
            return valor * 0.10;
        } else if (tipo.equals("ISS")) {
            return valor * 0.05;
        }
        return 0;
    }
}
```

Solução Guiada:

java

```
interface ImpostoStrategy {
    double calcular(double valor);
}

class ICMS implements ImpostoStrategy { /*...*/ }
class ISS implements ImpostoStrategy { /*...*/ }

class CalculadoraImposto {
    double calcular(ImpostoStrategy imposto, double valor) {
        return imposto.calcular(valor);
    }
}
```

Encerramento

- Discussão: Qual princípio SOLID é mais desafiador?

- **Próxima aula:** Tratamento de Exceções e Logging (Aula 5)

Tarefa:

- Refatorar um projeto pessoal aplicando Factory Method

Recursos:

- [Refactoring Guru - SOLID](#)
- [Java Design Patterns](#)

Dominar esses conceitos transformará seu código de "funciona" para "profissional"! Dúvidas? 😊

